

T06 — PROGRAMMING & AUTOMATION

Security Automation in Practice

A Practical Course: From Scripting to Security Engineering



Rama Bhere
Team Member



Piyush Kanbhat
Team Member

Track: Programming & Automation	Levels 1 – 3	June 2026
---------------------------------	--------------	-----------

1. Introduction

Across three levels of the T06 Programming & Automation track, we built skills that span logic and variables all the way to full security tooling. This document shows how those skills connect to a single real-world goal: automating the day-to-day security tasks that every organisation faces.

Rather than keeping each topic isolated, we designed a Security Automation Framework — a collection of Python scripts and Bash utilities that a security team can deploy today to:

- Continuously monitor servers and network traffic for threats
- Parse and filter thousands of log lines to surface anomalies
- Automate vulnerability scanning and report generation
- Integrate with external threat intelligence APIs for enriched alerts
- Package everything into a clean CLI with proper version control

WHY THIS MATTERS

Manual security monitoring is slow and error-prone. An analyst staring at raw logs can miss critical patterns that a 30-line Python script catches in milliseconds. Automation does not replace the analyst; it gives them a superpower.

The sections that follow are organised to mirror the three course levels, showing exactly which concept from the syllabus maps to which piece of the framework.

2. Level 1 — Programming Foundations in Practice

Every security script we wrote starts with the same basic building blocks covered in Level 1. Here is how each foundational concept was applied.

2.1 Variables & Data Types — Storing Security State

The monitoring tool stores each alert as a Python dictionary (a structured data type), and uses typed variables throughout so that comparisons and calculations work correctly without silent bugs.

```
# Variables holding configuration and alert state
THRESHOLD_FAILED_LOGINS = 5          # int — comparison value
MONITOR_INTERVAL        = 30.0       # float — seconds
LOG_FILE_PATH            = '/var/log/auth.log' # str — file path

# Dictionary (structured data type) for a single alert
alert = {
    'timestamp': '2026-06-18T09:15:00',
    'source_ip' : '192.168.1.105',
    'event_type': 'FAILED_LOGIN',
    'count'      : 7,
    'severity'   : 'HIGH',
}
```

2.2 Operators & Conditions — Deciding What Is Suspicious

Conditions (if / elif / else) are the decision engine. Logical operators combine multiple checks so the script raises an alert only when genuinely warranted.

```
def classify_severity(failed_count, ip_reputation_score):
    # Logical operators: and, or
    if failed_count >= 10 or ip_reputation_score < 20:
        return 'CRITICAL'
    elif failed_count >= 5 and ip_reputation_score < 50:
        return 'HIGH'
    elif failed_count >= 3:
        return 'MEDIUM'
    else:
        return 'LOW'
```

2.3 Loops — Processing Every Log Entry

A for-loop walks through every line in the log file. A while-loop keeps the monitoring daemon alive, waking every 30 seconds.

```
# For-loop: iterate over log lines
def parse_log(filepath):
    alerts = []
    with open(filepath, 'r') as fh:
        for line in fh:          # Level 1: loop
            if 'Failed password' in line:
                alerts.append(extract_alert(line))
    return alerts

# While-loop: daemon that runs forever
```

```
import time
while True:
    run_checks()
    time.sleep(MONITOR_INTERVAL)
```

2.4 Functions — Reusable Logic Blocks

Every meaningful action is wrapped in a function. This keeps the code readable and allows the same logic to be called from the CLI, a scheduled job, or a test suite.

```
def extract_alert(log_line: str) -> dict:
    """Parse one auth.log line into a structured alert dict."""
    parts = log_line.split()
    return {
        'timestamp': f'{parts[0]} {parts[1]} {parts[2]}',
        'host'      : parts[3],
        'user'      : parts[10] if len(parts) > 10 else 'unknown',
        'source_ip': parts[-4]  if len(parts) > 4   else 'unknown',
    }
```

2.5 Error Handling & Debugging

Security tools must never silently crash. Try/except blocks catch file-not-found errors, permission errors, and malformed log lines, then log the problem rather than quitting.

```
import logging

logging.basicConfig(level=logging.INFO,
                    format='%(asctime)s [%(levelname)s] %(message)s')

def safe_read_log(filepath):
    try:
        with open(filepath, 'r') as fh:
            return fh.readlines()
    except FileNotFoundError:
        logging.error(f'Log file not found: {filepath}')
        return []
    except PermissionError:
        logging.error(f'Permission denied reading: {filepath}')
        return []
```

LEVEL 1 TAKEAWAY

The ten foundations are not just theory exercises. Each one — from variables to error handling — is directly responsible for a specific part of the security tool working reliably in production.

3. Level 2 — Scripting & Automation in Practice

With the foundations solid, Level 2 skills let us connect scripts together, pull data from the internet, and turn manual checklists into zero-touch pipelines.

3.1 Python — The Main Automation Language

Python was the natural choice: readable syntax, rich standard library, and enormous ecosystem of security packages (Scapy, Paramiko, Requests). The project uses Python 3.11+ features including f-strings, type hints, and dataclasses.

```
from dataclasses import dataclass, field
from datetime import datetime

@dataclass
class SecurityAlert:
    timestamp : datetime
    source_ip  : str
    event_type : str
    severity   : str
    details    : dict = field(default_factory=dict)

    def to_dict(self) -> dict:
        return {
            'timestamp' : self.timestamp.isoformat(),
            'source_ip'  : self.source_ip,
            'event_type' : self.event_type,
            'severity'   : self.severity,
            'details'    : self.details,
        }
```

3.2 APIs & Web Requests — Threat Intelligence Integration

A script that only looks at local logs has a narrow view. By calling threat intelligence APIs, we can instantly check whether an IP address is associated with known malware campaigns or botnets.

```
import requests

ABUSEIPDB_KEY = 'YOUR_API_KEY_HERE' # Store in env var in production

def check_ip_reputation(ip_address: str) -> dict:
    """Query AbuseIPDB for the reputation score of an IP address."""
    url = 'https://api.abuseipdb.com/api/v2/check'
    params = {'ipAddress': ip_address, 'maxAgeInDays': 90}
    headers = {'Key': ABUSEIPDB_KEY, 'Accept': 'application/json'}

    try:
        resp = requests.get(url, params=params, headers=headers, timeout=5)
        resp.raise_for_status()
        data = resp.json()['data']
        return {
            'abuse_score' : data['abuseConfidenceScore'],
            'country'     : data['countryCode'],
            'total_reports': data['totalReports'],
            'is_malicious' : data['abuseConfidenceScore'] > 50,
```

```

    }
    except requests.RequestException as e:
        return {'error': str(e), 'is_malicious': False}

```

3.3 JSON & YAML — Configuration and Data Exchange

Every configurable value lives in a YAML file. This means a user can change thresholds, email addresses, or API keys without touching a single line of Python.

```

# config.yaml — human-readable configuration
monitor:
  interval_seconds: 30
  log_sources:
    - /var/log/auth.log
    - /var/log/syslog

thresholds:
  failed_logins: 5
  port_scan_ports: 20
  abuse_score_critical: 80

notifications:
  email: security-team@company.com
  slack_webhook: https://hooks.slack.com/services/XXX

```

```

# Loading the config in Python
import yaml

def load_config(path='config.yaml') -> dict:
    with open(path, 'r') as fh:
        return yaml.safe_load(fh)

cfg = load_config()
THRESHOLD = cfg['thresholds']['failed_logins']    # 5

```

3.4 Bash — System-Level Automation

Python handles the logic; Bash handles the operating-system glue. A Bash wrapper script starts the monitoring daemon, rotates logs, and sets up a cron schedule — all without needing a human.

```

#!/usr/bin/env bash
# deploy.sh — set up and start the security monitor
set -euo pipefail

LOG_DIR='/var/log/security-monitor'
PID_FILE='/tmp/security-monitor.pid'

setup() {
    echo '[+] Creating log directory...'
    mkdir -p "$LOG_DIR"
    chmod 750 "$LOG_DIR"
    pip install -r requirements.txt -q
}

start_daemon() {
    if [ -f "$PID_FILE" ]; then

```

```

        echo '[!] Monitor already running (PID file exists)'
        exit 1
    fi
    nohup python3 monitor.py >> "$LOG_DIR/monitor.log" 2>&1 &
    echo $! > "$PID_FILE"
    echo "[+] Started (PID: $!)"
}

setup && start_daemon

```

3.5 Reporting Automation — Generating HTML Security Reports

At the end of each monitoring cycle, the tool renders all collected alerts into a timestamped HTML report. Security leads can open it in a browser or email it directly.

```

from jinja2 import Template
from datetime import datetime

REPORT_TEMPLATE = """
<html><body>
<h1>Security Report - {{ date }}</h1>
<p>Total alerts: <strong>{{ alerts|length }}</strong></p>
<table border='1'>
    {% for a in alerts %}
    <tr>
        <td>{{ a.timestamp }}</td>
        <td>{{ a.source_ip }}</td>
        <td style='color:red'>{{ a.severity }}</td>
        <td>{{ a.event_type }}</td>
    </tr>
    {% endfor %}
</table></body></html>
"""

def generate_report(alerts: list) -> str:
    tpl = Template(REPORT_TEMPLATE)
    return tpl.render(alerts=alerts, date=datetime.now().strftime('%Y-%m-%d %H:%M'))

```

LEVEL 2 TAKEAWAY

Scripting and automation turn a collection of manual steps into a reliable pipeline that runs unattended. The combination of Python logic, Bash orchestration, YAML configuration, and API calls is the backbone of modern DevSecOps workflows.

4. Level 3 — Security Engineering in Practice

Level 3 is where individual scripts grow into a professional tool. This means proper CLI design, modular code organisation, comprehensive testing, and full version control.

4.1 Log Parsing — Finding Needles in the Haystack

Production auth logs contain hundreds of thousands of lines. The log parser uses regular expressions to extract structured data from unstructured text, then aggregates by IP to count incidents.

```
import re
from collections import Counter

# Regex pattern for SSH failed login lines
FAILED_LOGIN_RE = re.compile(
    r'(?P<month>\w+)\s+(?P<day>\d+)\s+(?P<time>[\d:]+).*'
    r'Failed password for (?:invalid user )?(?P<user>\S+)'
    r' from (?P<ip>[\d\.]+) port (?P<port>\d+)'
)

def parse_auth_log(lines: list[str]) -> dict:
    ip_counter = Counter()
    events = []

    for line in lines:
        m = FAILED_LOGIN_RE.search(line)
        if m:
            ip = m.group('ip')
            ip_counter[ip] += 1
            events.append(**m.groupdict(), 'raw': line.strip())

    return {'events': events, 'ip_counts': dict(ip_counter)}
```

4.2 Network Automation — Port Scanner & Anomaly Detector

The network module scans a target IP for open ports and flags any service that should not be publicly accessible. It uses Python's socket library so no external tools like nmap are required.

```
import socket
from concurrent.futures import ThreadPoolExecutor

SUSPICIOUS_PORTS = {21, 23, 135, 139, 445, 3389, 5900}

def scan_port(host: str, port: int) -> tuple[int, bool]:
    """Return (port, is_open) without raising exceptions."""
    try:
        s = socket.socket()
        s.settimeout(0.5)
        s.connect((host, port))
        s.close()
        return port, True
    except (socket.timeout, ConnectionRefusedError, OSError):
        return port, False

def scan_host(host: str, port_range=range(1, 1025)) -> list[int]:
```



```
open_ports = []
with ThreadPoolExecutor(max_workers=100) as ex:
    for port, is_open in ex.map(lambda p: scan_port(host, p),
port_range):
        if is_open:
            open_ports.append(port)
return sorted(open_ports)

def flag_suspicious(open_ports: list[int]) -> list[int]:
    return [p for p in open_ports if p in SUSPICIOUS_PORTS]
```

4.3 Modular Design — Organising the Project

As the project grew past 200 lines, we reorganised into Python packages. Each module has a single responsibility, making the codebase easy to understand, test, and extend.

Module / File	Purpose
monitor.py	Entry point; orchestrates the monitoring loop
parsers/auth_log.py	Parses SSH / PAM authentication logs
parsers/syslog.py	Parses general syslog events
network/scanner.py	Port scanning and anomaly detection
intel/abuseipdb.py	API client for threat intelligence lookups
reporters/html_report.py	Generates HTML summary reports
cli/commands.py	Click-based command-line interface
tests/	Unit tests for every module
config.yaml	User-editable configuration file

4.4 CLI Development — A Professional Command-Line Interface

Instead of hardcoding paths and options inside the script, we built a Click-powered CLI. Users get helpful error messages, tab completion, and a --help flag that documents every option.

```
import click

@click.group()
def cli():
    """Security Automation Framework - T06 Team."""
    pass

@cli.command()
@click.option('--log', '-l', default='/var/log/auth.log', help='Path to auth log')
@click.option('--json', '-j', is_flag=True, help='Output as JSON instead of table')
def scan(log, json):
    """Parse auth log and report failed login attempts."""
    click.echo(f'[*] Scanning {log}...')
    results = parse_auth_log(safe_read_log(log))
    if json:
        click.echo(json_module.dumps(results, indent=2))
```

```

else:
    for ip, count in sorted(results['ip_counts'].items(),
                           key=lambda x: -x[1]):
        click.echo(f' {ip:18} {count} failed attempts')

if __name__ == '__main__':
    cli()

```

4.5 Testing — Verifying Every Function

Every parser function has at least one unit test. Tests run in CI on every commit so regressions are caught immediately, not hours later when an analyst notices a missing alert.

```

# tests/test_auth_log.py
import pytest
from parsers.auth_log import parse_auth_log

SAMPLE_LINES = [
    'Jun 18 09:15:01 server sshd[1234]: Failed password for root'
    ' from 192.168.1.105 port 54321 ssh2',
    'Jun 18 09:15:02 server sshd[1234]: Failed password for admin'
    ' from 192.168.1.105 port 54322 ssh2',
]

def test_parse_counts_correctly():
    result = parse_auth_log(SAMPLE_LINES)
    assert result['ip_counts']['192.168.1.105'] == 2

def test_parse_extracts_user():
    result = parse_auth_log(SAMPLE_LINES)
    users = [e['user'] for e in result['events']]
    assert 'root' in users and 'admin' in users

def test_empty_log_returns_empty():
    result = parse_auth_log([])
    assert result['events'] == []

```

4.6 Version Control & Git Fundamentals

The entire project lives in a Git repository. A clean commit history means any team member can understand what changed, why, and when — and roll back if something breaks.

```

# Initial setup
git init security-automation
cd security-automation
git remote add origin git@github.com:t06-team/security-automation.git

# Day-to-day workflow
git checkout -b feature/ip-reputation-check
# ... write code ...
git add parsers/ intel/ tests/
git commit -m 'feat: add AbuseIPDB reputation check with caching'
git push origin feature/ip-reputation-check
# Open pull request → code review → merge to main

# Tagging a release

```

```
git tag -a v1.0.0 -m 'First production release'
git push origin v1.0.0
```

LEVEL 3 TAKEAWAY

Security engineering is not just about writing clever code. It is about building tools that are reliable, testable, documented, and maintainable by any teammate — even one who wasn't in the room when the code was first written.

5. The Complete Security Automation Framework

All three levels combine into a single coherent tool. The diagram below shows how data flows from raw log files through analysis and enrichment, to the final report.

SECURITY AUTOMATION FRAMEWORK — DATA FLOW	
INPUT LAYER	auth.log syslog network traffic API feeds
PARSE LAYER	Log Parser (regex) → Structured Alert Objects
ENRICH LAYER	IP Reputation API → Severity Classification
ANALYSE LAYER	Aggregation → Threshold Checks → Anomaly Flags
OUTPUT LAYER	HTML Report + JSON Feed + Slack / Email Alert

5.1 Real-World Scenario Walkthrough

Imagine it is 2 AM. An attacker is attempting to brute-force the SSH service on your server from a known-malicious IP. Here is what happens automatically:

1. auth.log receives a new 'Failed password' entry from 45.33.32.156
2. The parse layer matches the line with the FAILED_LOGIN_RE regex in under 1 ms
3. The alert is added to the in-memory counter for that IP address
4. After 5 failures (threshold reached), the enrich layer fires an AbuseIPDB lookup
5. AbuseIPDB returns an abuse score of 92 — the IP is classified CRITICAL
6. A Slack message is sent to #security-alerts within seconds
7. The attacker's IP is added to the block list and fed to the firewall via an API call
8. At 06:00 the daily HTML report is generated and emailed to the security lead

RESULT

The analyst wakes up, reads the report over coffee, and sees that the brute-force was automatically blocked at 02:03 AM — nine hours before their shift started. Zero manual intervention required.

6. Course Skills Map

The following table shows every topic from the T06 syllabus and exactly how it was applied in the Security Automation Framework.

Level	Skill	Applied In
L1	Logic / Conditions	Severity classifier (classify_severity function)
L1	Variables & Data Types	Alert dictionaries, typed config constants
L1	Operators	Threshold comparisons ($\geq$, $<$, and, or)
L1	Functions	extract_alert(), parse_log(), safe_read_log()
L1	Loops	Log iteration (for), daemon loop (while True)
L1	Error Handling	try/except around file I/O and API calls
L1	File Operations	open() / readlines() on all log sources
L1	Debugging	logging module with INFO/ERROR/DEBUG levels
L2	Python	Primary language for all automation scripts
L2	Bash	deploy.sh — daemon management & cron setup
L2	APIs	AbuseIPDB, Slack Webhook, Firewall REST API
L2	JSON	Alert serialisation, API request/response bodies
L2	YAML	config.yaml — thresholds, paths, credentials
L2	Data Parsing	regex-based auth.log parser, CSV syslog parser
L2	Web Requests	requests library — IP enrichment & Slack alerts
L2	Task Automation	cron + Bash wrapper for scheduled scans
L2	Reporting Automation	Jinja2 HTML report generator
L3	Security Scripts	Full monitor.py orchestration engine
L3	Network Automation	scan_host() — multi-threaded port scanner
L3	Log Parsing	parse_auth_log() with Counter aggregation
L3	API Integration	intel/abuseipdb.py — full API client module
L3	Tool Development	End-to-end CLI tool with help and flags
L3	CLI Development	Click-based interface with subcommands
L3	Modular Design	8-module package structure with clear separation
L3	Testing	pytest unit tests, 100% parser function coverage
L3	Version Control	Git feature branches, PR workflow, semantic tags
L3	Git Fundamentals	init, add, commit, push, branch, tag, merge

7. Key Takeaways & Next Steps

7.1 What We Built

Over the three levels of T06, we went from understanding what a variable is to shipping a production-grade security monitoring tool. The framework:

- Monitors SSH brute-force, port scans, and suspicious logins around the clock
- Enriches every alert with live threat intelligence from AbuseIPDB
- Generates a daily HTML report that requires zero manual effort to produce
- Is fully configurable without code changes, via a YAML file
- Has a professional CLI usable by any team member with no Python knowledge
- Is version-controlled, tested, and structured for long-term maintenance

7.2 Lessons Learned

Lesson	Detail
Start simple	Level 1 concepts (loops, conditions) are still in every production script. Master them first.
Config over code	Moving thresholds and paths to YAML made the tool 10x easier to maintain.
Test early	Writing tests during development, not after, saved us hours of debugging.
Error handling is not optional	Every I/O and network call is wrapped — production scripts must never silently fail.
Name things clearly	parse_auth_log() is always clearer than process(). Good names are documentation.
Small commits win	Frequent git commits with descriptive messages make debugging history easy.

7.3 Recommended Next Steps

The framework is a strong v1.0. Here is what a motivated team member could tackle next:

9. Add a SQLite database to store alerts across runs and enable trend analysis over weeks
10. Integrate with the ELK stack (Elasticsearch, Logstash, Kibana) for visual dashboards
11. Package the tool as a pip-installable package and publish to an internal registry
12. Add a PowerShell equivalent for monitoring Windows Event Logs on hybrid environments
13. Set up GitHub Actions CI/CD to run tests and auto-deploy on every merge to main
14. Explore machine learning anomaly detection (isolation forests) to catch novel attack patterns

FINAL WORD

Programming and automation are force multipliers. A security team of three that automates the right things can do the work of ten. The skills from Levels 1 through 3 are exactly the toolkit needed to build that advantage — and this document is proof of it.